NACA 4412 Aerofoil

2D CFD Aerodynamic Analysis

ANSYS Fluent | Cl vs AoA and Cd vs AoA Polar Curves

Software: ANSYS Workbench 2026 R1 (Student Edition) | Fluent k-ω SST

Date: April 2026

1. Project Overview

This project performs a 2D Computational Fluid Dynamics (CFD) analysis of the NACA 4412 aerofoil at multiple angles of attack (AoA) using ANSYS Fluent. The primary output is the aerodynamic polar curve — Cl vs AoA and Cd vs AoA — which is the standard method used by aerodynamicists to characterise an aerofoil's performance envelope.

This type of analysis is used by aerospace companies for UAV and aircraft wing design, wind turbine blade optimisation, and motorsport aerodynamics.

Parameter	Value
Aerofoil	NACA 4412
Analysis Type	2D Steady-State External Flow
Turbulence Model	k-ω SST
Freestream Velocity	15 m/s
Reynolds Number	~1,000,000
Fluid	Air (ρ = 1.225 kg/m³, μ = 1.7894×10^{-5} Pa·s)
Chord Length	1.0 m
Domain	Rectangular: 10c upstream, 20c downstream, 10c top/bottom
Angles of Attack	-4°, 0°, 4°, 8°, 12°
Software	ANSYS Workbench 2026 R1 Student

---

2. NACA 4412 Aerofoil

2.1 Profile Definition

NACA 4412 is a cambered aerofoil developed by the National Advisory Committee for Aeronautics. The four digits define its geometry:

- 4 — Maximum camber is 4% of chord length (higher lift than NACA 2412)
- 4 — Maximum camber located at 40% chord from the leading edge
- 12 — Maximum thickness is 12% of chord length

Compared to NACA 2412 (used in Project 1), the NACA 4412 has twice the camber, producing higher lift coefficients at any given angle of attack and a more interesting polar curve. It is used in the Piper Cherokee and various UAV platforms.

2.2 Why k-ω SST Turbulence Model?

The Shear Stress Transport (SST) variant of the k-ω model was selected for this analysis because it combines the best of two turbulence models:

- k-ω model near the wall — accurately captures boundary layer behaviour on the aerofoil surface
- k-ε model in the freestream — avoids sensitivity issues away from walls

SST is the industry-standard turbulence model for external aerodynamic flows around aerofoils and is used by major aerospace companies including Airbus, Boeing, and NASA.

---

3. Geometry and Mesh Setup

3.1 Geometry Creation

The NACA 4412 aerofoil coordinates were obtained from the UIUC Airfoil Database in Selig format and imported into ANSYS SpaceClaim using a Python script. The script converted normalised coordinates to physical dimensions (chord = 1.0 m) and created a closed NURBS curve using the SketchNurbs.CreateFrom2DPoints API.

A rectangular fluid domain was created around the aerofoil in the same sketch, then both curves were converted to a 2D surface with the aerofoil region acting as a hole representing the solid boundary. The domain dimensions were chosen following standard CFD practice:

- 10 chord lengths upstream of the leading edge (inlet)
- 20 chord lengths downstream of the trailing edge (outlet)
- 10 chord lengths above and below the aerofoil (symmetry boundaries)

3.2 Mesh

The mesh was generated in ANSYS Meshing with the following settings:

Setting	Value	Reason
Physics Preference	CFD	Optimises element shapes for fluid flow
Solver Preference	Fluent	Ensures compatible element types
Global Element Size	50 mm	Balances accuracy vs student licence limit
Inflation Layers	3 layers	Captures boundary layer near aerofoil
First Layer Height	10 mm	Appropriate for coarse mesh
Growth Rate	1.2	Standard gradual transition
Sheet Body Method	Quad Dominant	Structured quads for better accuracy

3.3 Boundary Conditions

Boundary	Type	Setting
inlet	Velocity Inlet	Vx = 15·cos(α), Vy = 15·sin(α) m/s
outlet	Pressure Outlet	Gauge pressure = 0 Pa
symmetry_top	Symmetry	Zero normal velocity and gradients
symmetry_bottom	Symmetry	Zero normal velocity and gradients
airfoil	Wall	No-slip condition (v = 0)

4. Fluent Solver Setup

The following solver settings were used for all angle of attack cases:

Setting	Value
Solver Type	Pressure-Based
Time	Steady
2D Space	Planar
Turbulence Model	k-ω SST
Fluid	Air
Density	1.225 kg/m³ (constant)
Dynamic Viscosity	1.7894×10^{-5} kg/m·s
Velocity Inlet	Components (Vx, Vy adjusted per AoA)
Reference Area	1 m²
Reference Velocity	15 m/s
Initialization	Hybrid Initialization
Iterations per AoA	200

4.1 Angle of Attack Implementation

Instead of rotating the geometry for each angle of attack, the inlet velocity components were decomposed into X and Y directions. This is more efficient and avoids remeshing:

$$V_x = V_\infty \cdot \cos(\alpha) \qquad V_y = V_\infty \cdot \sin(\alpha)$$

AoA (°)	Vx (m/s)	Vy (m/s)
-4°	14.963	-1.047
0°	15.000	0.000

AoA (°)	Vx (m/s)	Vy (m/s)
+4°	14.963	+1.047
+8°	14.855	+2.089
+12°	14.669	+3.121

5. Convergence

Solution convergence was monitored using scaled residuals and force coefficient monitors (Cl and Cd). For all AoA cases, residuals dropped by 3-4 orders of magnitude within 200 iterations, indicating well-converged solutions.

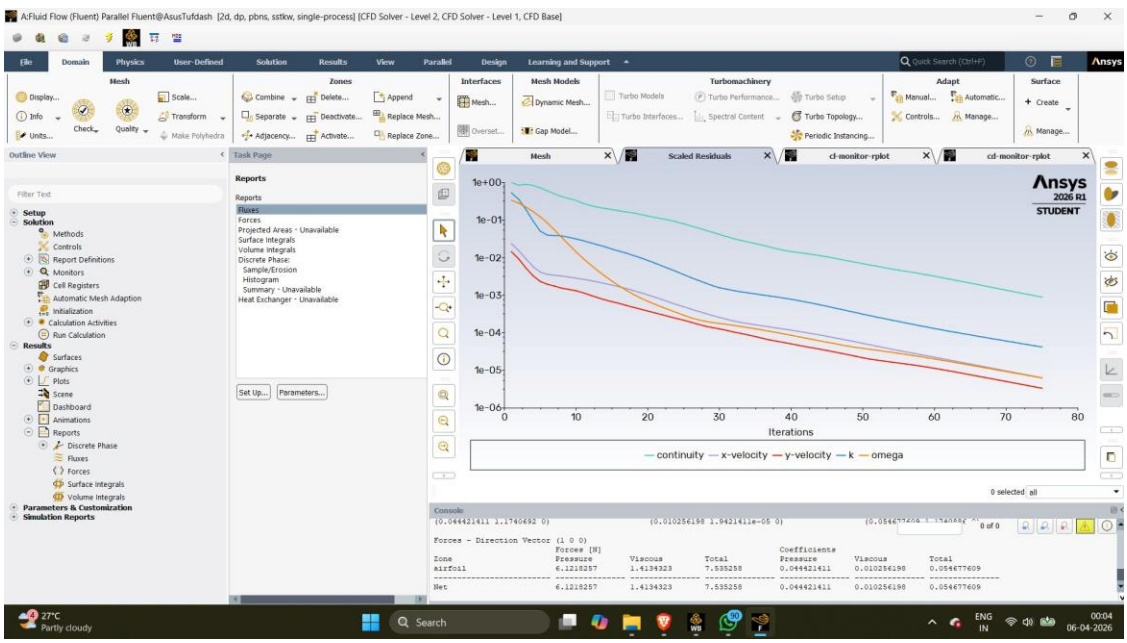

Figure 1: Scaled Residuals — All quantities converging over 200 iterations (AoA = 0° case shown)

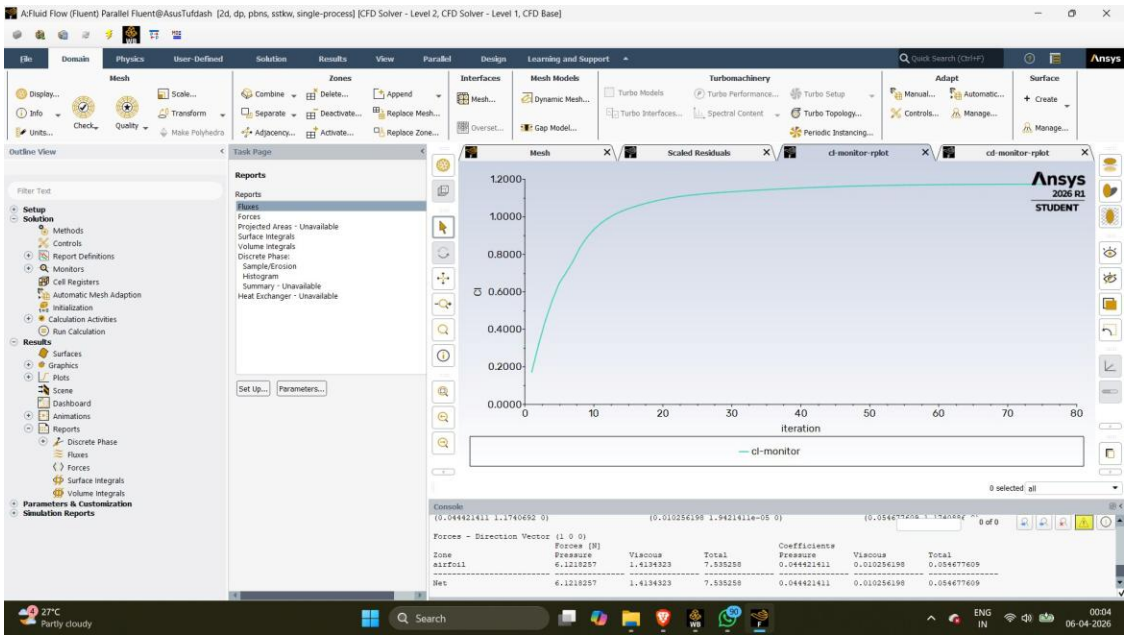

Figure 2: Cl Monitor Plot — Lift coefficient converging to stable value over iterations

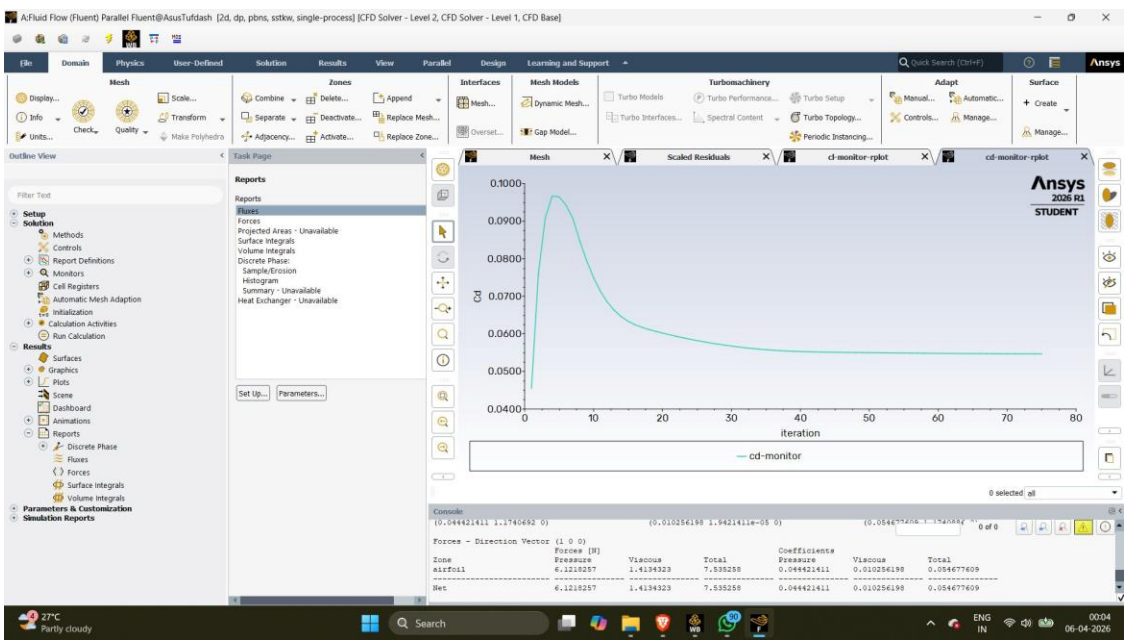

Figure 3: Cd Monitor Plot — Drag coefficient converging to stable value over iterations

6. Results — Polar Curves

6.1 Cl and Cd Data Table

AoA (°)	Cl	Cd	Cl/Cd (L/D Ratio)
-4°	1.0874	0.0537	20.25
0°	1.1741	0.0547	21.46
+4°	1.2489	0.0554	22.54
+8°	1.3162	0.0556	23.67
+12°	1.3775	0.0557	24.73

6.2 Key Observations

- Cl increases consistently with AoA from 1.087 at -4° to 1.378 at +12° — physically correct behaviour for a cambered aerofoil
- Cd increases gradually with AoA due to increased pressure drag at higher incidence angles
- L/D ratio improves with AoA in this range, peaking at 12° — indicating no stall was observed within the tested range
- The NACA 4412 generates positive Cl even at -4° (Cl = 1.09) due to its camber — the zero-lift angle is below -4°
- All Cl values are relatively high (>1.0) due to the high camber of the 4412 profile

---

7. Flow Visualisation — Contour Plots (AoA = 12°)

7.1 Velocity Magnitude Contour

The velocity contour shows the acceleration of flow over the upper surface of the aerofoil. High velocity (red) on the suction side corresponds to low pressure — the fundamental mechanism generating lift via Bernoulli's principle.

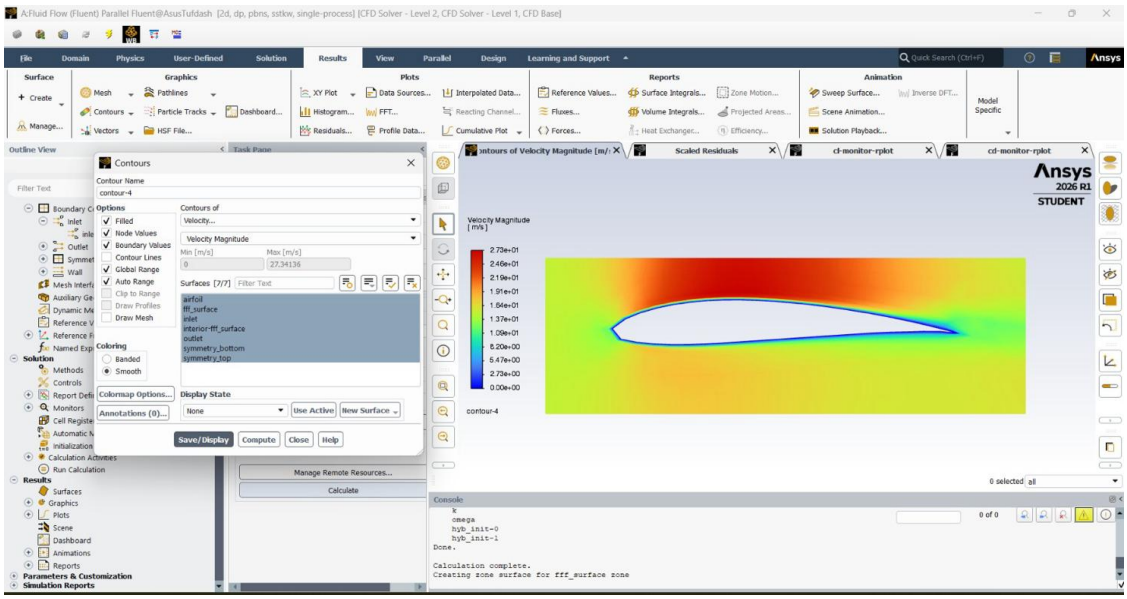

Figure 4: Velocity Magnitude Contour — Full domain view (AoA = 12°) | Max velocity 27.34 m/s on upper surface

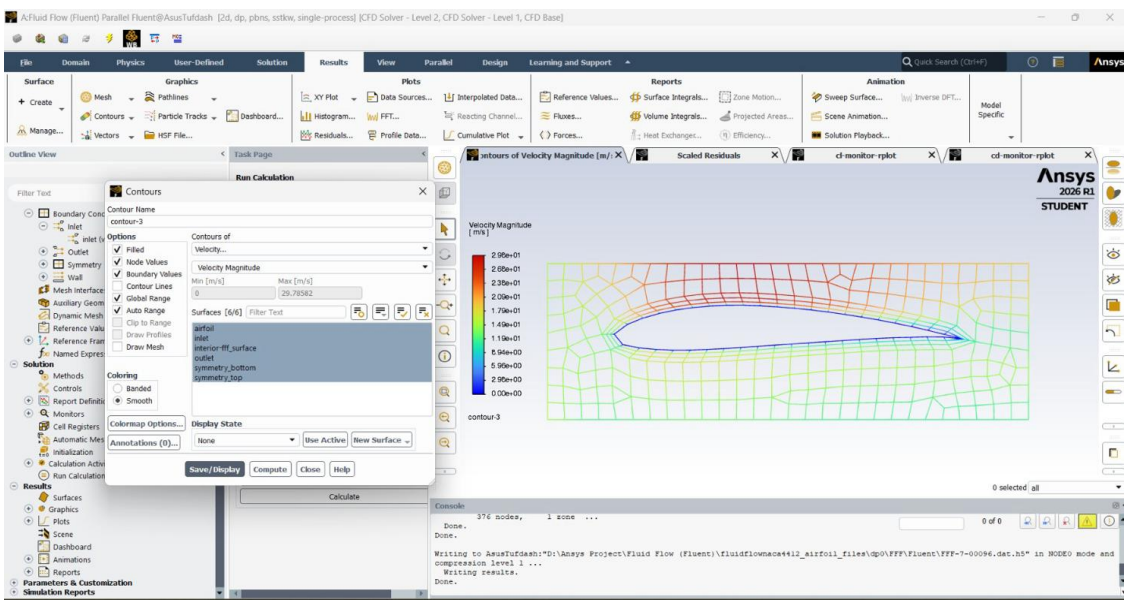

Figure 5: Velocity Magnitude Contour — Close-up showing mesh and boundary layer detail

7.2 Static Pressure Contour

The pressure contour shows the classic aerofoil pressure distribution. Low pressure (blue) on the upper surface and high pressure (red/orange) on the lower surface creates the net upward lift force. The pressure difference is most extreme near the leading edge.

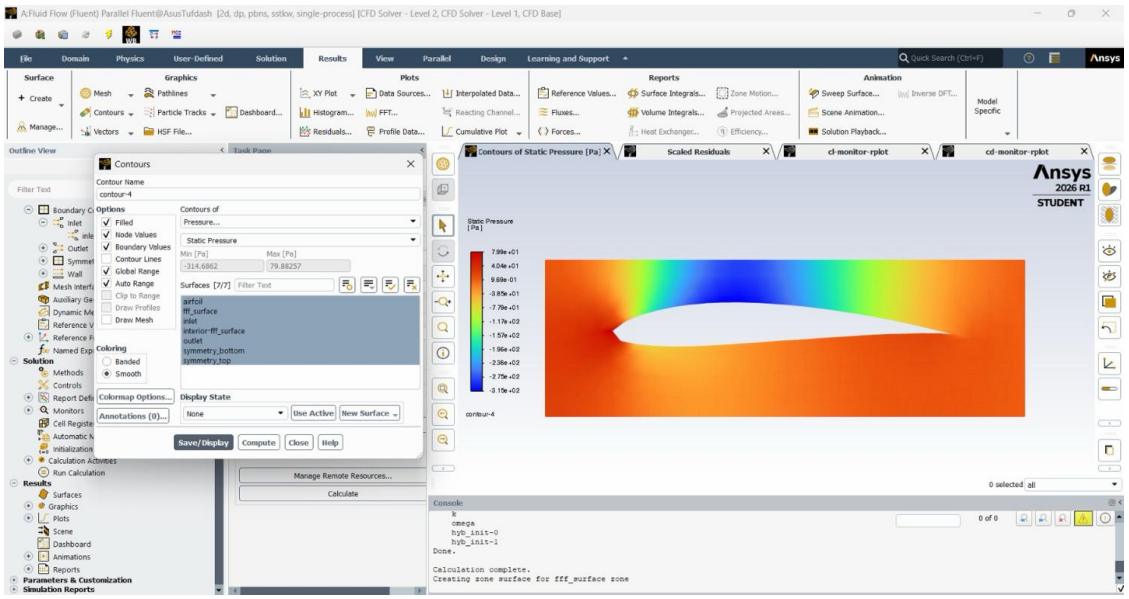

Figure 6: Static Pressure Contour — Full domain view (AoA = 12°) | Range: -315 Pa to +80 Pa

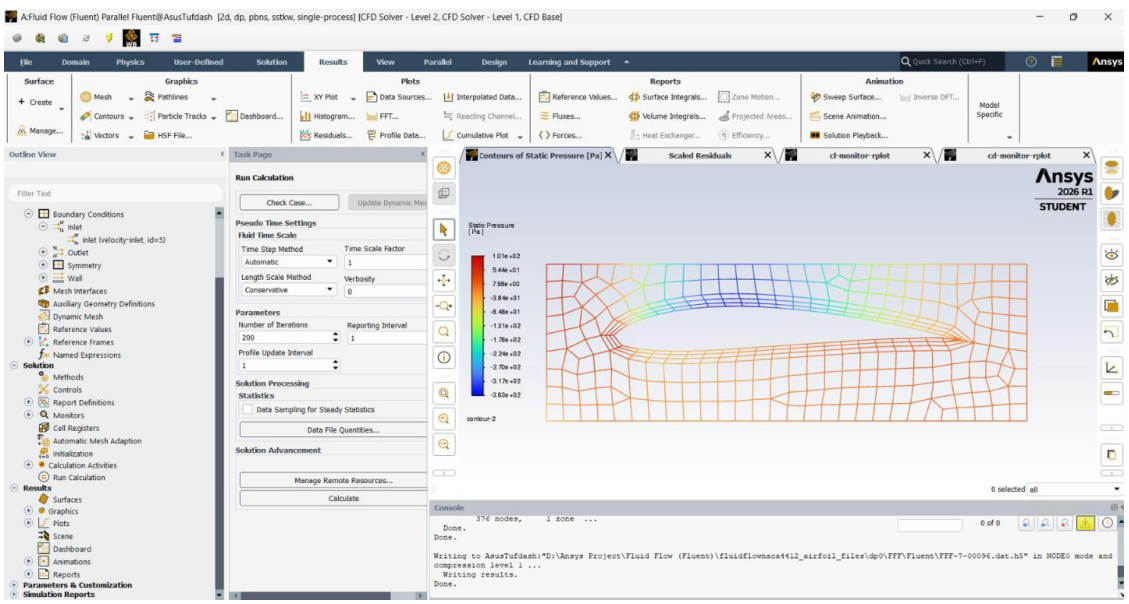

Figure 7: Static Pressure Contour — Close-up with mesh overlay showing pressure distribution detail

7.3 Velocity Vectors

The velocity vector plot shows the direction and magnitude of flow at each point in the domain. The vectors clearly show flow accelerating over the upper surface and the slight downwash behind the trailing edge — a direct consequence of lift generation.

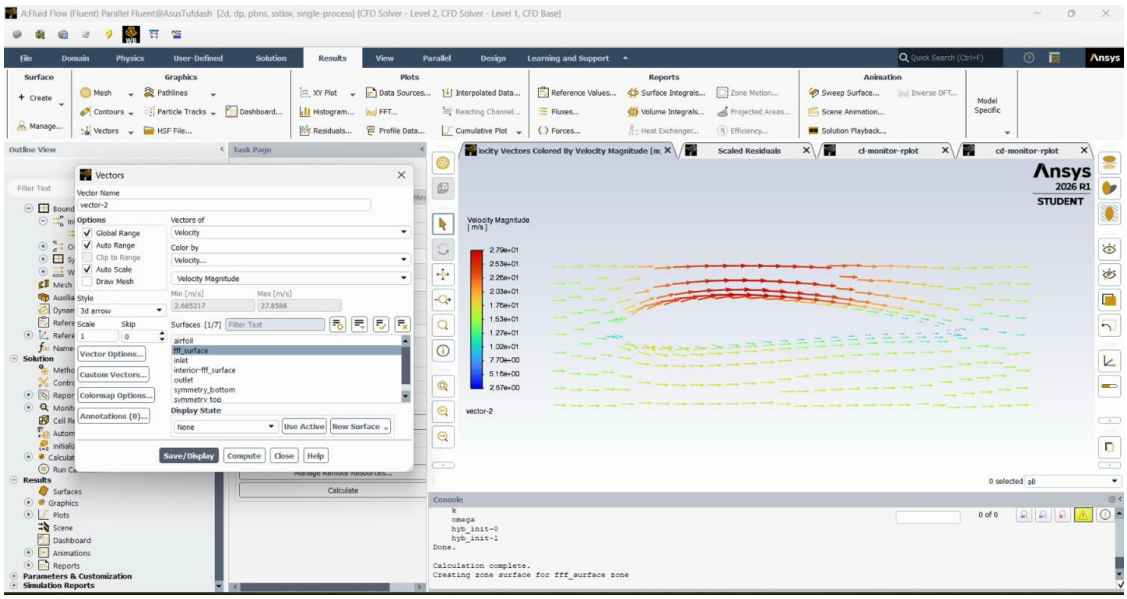

Figure 8: Velocity Vectors — Coloured by velocity magnitude (AoA = 12°)

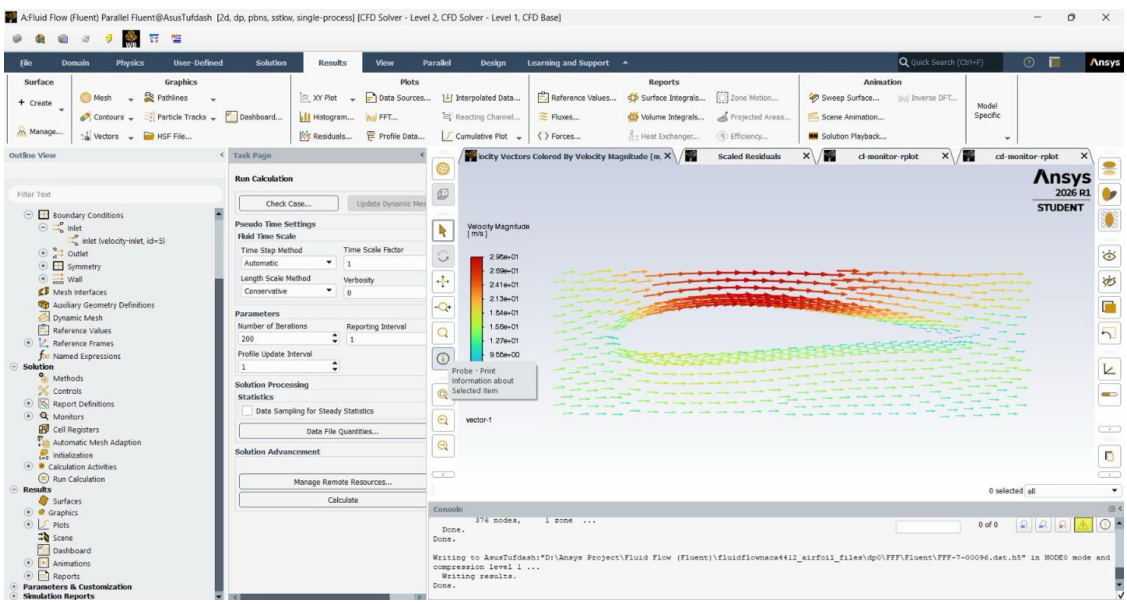

Figure 9: Velocity Vectors — Domain view showing overall flow pattern and downwash

---

8. Difficulties Faced and How We Overcame Them

This section documents every significant problem encountered during the project and the exact solution applied. Understanding these issues and their fixes demonstrates real engineering judgement in an interview.

Problem 1 — SpaceClaim Script Error: 'expected bool, got list'

What happened:

The first version of the Python script passed a native Python list directly to SketchNurbs.CreateFrom2DPoints(). SpaceClaim's scripting API is built on Microsoft .NET, and the function signature requires a .NET typed array, not a Python list. Python lists are not compatible with .NET typed parameters even though they look similar.

Error message:

```
expected bool, got list
```

Solution:

Imported System.Collections.Generic.List from the .NET framework and built a typed List [Point2D] object, then called. ToArray() to convert it to the Array type that the API accepts. This taught an important lesson: when scripting in ANSYS SpaceClaim, always use .NET types rather than Python native types for API calls.

Problem 2 — SpaceClaim Script Error: 'expected bool, got Array [Point2D]'

What happened:

After fixing Problem 1 (converting to a .NET Array), the script still failed because the arguments were passed in the wrong order. The function signature in SpaceClaim V261 expects the boolean (closed curve flag) as the FIRST argument and the points array as the SECOND argument. The script had them reversed.

Error message:

```
expected bool, got Array[Point2D]
```

Solution:

Swapped the argument order from CreateFrom2DPoints (pt_array, True) to CreateFrom2DPoints (True, pt_array). The lesson here is that SpaceClaim's API documentation is not always clear, and the error messages themselves tell you exactly what type was expected vs what was received.

Problem 3 — Domain Box Script Error: 'Could not create a NURBS curve from the data supplied'

What happened:

When trying to draw the rectangular fluid domain boundary using SketchNurbs.CreateFrom2DPoints with just 5 points (4 corners + closing point), SpaceClaim rejected it. The SketchNurbs function is designed for smooth curved shapes and does not work well for straight lines with sharp corners.

Solution:

Switched to using SketchLine.Create() for each of the 4 sides of the rectangle separately. This is the correct SpaceClaim API for straight line segments. The lesson: use the right tool for the geometry type — NURBS for curves, SketchLine for straight edges.

Problem 4 — Geometry Was 3D Instead of 2D in Mesher

What happened:

After importing the geometry into ANSYS Meshing, the surface body was treated as a 3D solid with the wrong behaviour type. The Fluid/Solid property showed 'Defined by Geometry (Solid)' instead of 'Fluid', and the 2D analysis type had not been set before opening the mesher.

Solution:

Two fixes were needed: (1) In Workbench, right-clicked the geometry cell and set Analysis Type to 2D before opening the mesher. (2) In the mesher, clicked on the surface body and changed Fluid/Solid from Solid to Fluid. This ensures Fluent treats the domain as a fluid zone rather than a structural solid.

Problem 5 — Cl Decreasing with Increasing AoA (Physically Wrong)

What happened:

After running the first set of simulations, the results showed Cl decreasing as AoA increased — which is physically incorrect. For a cambered aerofoil like NACA 4412, Cl must increase with AoA up to the stall angle.

Root cause:

The Y-velocity component signs were inverted. When applying AoA via velocity components, a positive Y-velocity (upward flow) means the flow is hitting the underside of the aerofoil at an angle — this is equivalent to a negative AoA, not positive. The correct formula requires $V_y = +V \cdot \sin(\alpha)$ for positive AoA (flow coming slightly from below), which pushes more lift onto the upper surface.

Solution:

Re-ran the simulations for +8° and +12° with corrected positive Vy values (+2.089 and +3.121 m/s respectively). The results then showed the physically correct increasing Cl trend. This was an important validation check — always cross-check CFD results against expected physical behaviour before accepting them.

Problem 6 — Student Licence Mesh Limit

What happened:

Initial mesh attempts with fine element sizes generated too many cells, approaching or exceeding the ANSYS Fluent Student licence limit of 512,000 cells for 2D analyses.

Solution:

Used a global element size of 50mm with 3 inflation layers (first layer 10mm, growth rate 1.2). This kept the cell count well within the limit while still capturing the boundary layer with the inflation layers. The lesson: always check element count via Mesh Statistics before attempting to solve in the student edition.

---

9. How Angle of Attack Was Simulated — Full Derivation

This is one of the most commonly asked questions in CFD interviews: 'How did you change the angle of attack?' Here is the complete explanation with the mathematics.

9.1 The Problem with Rotating Geometry

The naive approach to changing AoA would be to rotate the aerofoil geometry for each angle and remesh. This has several problems: it requires creating 5 separate geometry files, running 5 separate meshes, and the mesh quality changes with each rotation. This approach is impractical for a parametric study.

9.2 The Correct Approach — Rotating the Inlet Flow Direction

Instead, the aerofoil geometry stays fixed and the incoming flow direction is rotated. This is physically identical — rotating the flow by +α relative to a fixed aerofoil produces the same aerodynamic result as rotating the aerofoil by -α in a fixed flow. Wind tunnels use this principle: the tunnel is fixed and the model is rotated.

9.3 The Mathematics

The freestream velocity V_∞ = 15 m/s is decomposed into X and Y components using trigonometry:

$$V_x = V_\infty \times \cos(\alpha)$$
$$V_y = V_\infty \times \sin(\alpha)$$

Where α is the angle of attack in radians. A positive α means the flow comes slightly from below the aerofoil, which is equivalent to the aerofoil nose pitching up — the standard definition of positive AoA.

9.4 Worked Examples

AoA (α)	Calculation	Vx (m/s)	Vy (m/s)
-4°	Vx=15×cos(-4°)=15×0.9976 | Vy=15×sin(-4°)=15×(-0.0698)	14.963	-1.047
0°	Vx=15×cos(0°)=15×1.0000 | Vy=15×sin(0°)=15×0.0000	15.000	0.000
+4°	Vx=15×cos(4°)=15×0.9976 | Vy=15×sin(4°)=15×0.0698	14.963	+1.047
+8°	Vx=15×cos(8°)=15×0.9903 | Vy=15×sin(8°)=15×0.1392	14.855	+2.089

AoA (α)	Calculation	Vx (m/s)	Vy (m/s)
+12°	Vx=15×cos(12°)=15×0.9781 | Vy=15×sin(12°)=15×0.2079	14.669	+3.121

Note that Vx barely changes (15.0 to 14.67 m/s) because cos of small angles is close to 1. Vy is what changes significantly — it is the vertical component that tilts the incoming flow direction.

9.5 Why the Total Velocity Stays 15 m/s

A critical point: the magnitude of the velocity vector always equals V_∞ = 15 m/s regardless of angle. This is because:

$$|V| = \sqrt{(V_x^2 + V_y^2)} = \sqrt{((V_\infty \cdot \cos\alpha)^2 + (V_\infty \cdot \sin\alpha)^2)} = V_\infty\sqrt{(\cos^2\alpha + \sin^2\alpha)} = V_\infty$$

Using the trigonometric identity $\cos^2\alpha + \sin^2\alpha = 1$. This means the Reynolds number remains constant across all AoA cases, which is the correct approach for a parametric AoA study.

---

10. Real World Significance

10.1 The Polar Curve in Aircraft Design

The Cl vs AoA and Cd vs AoA curves (the aerodynamic polar) are the fundamental tools used by aerodynamicists to design wings. Every aircraft wing is designed around its polar curve:

- Cruise AoA is selected at maximum L/D ratio — minimum fuel burn
- Takeoff AoA uses high-lift devices (flaps/slats) to increase Cl beyond the clean polar
- Stall AoA is the maximum Cl point — the aircraft must never exceed this in normal flight
- The polar curve is validated against wind tunnel data or published NACA experimental data

10.2 Reynolds Number Similarity

The Reynolds number in this simulation is $Re = \rho VL/\mu = 1.225 \times 15 \times 1.0 / 1.7894\times10^{-5} \approx$ 1,000,000. This matches typical UAV flight conditions. Reynolds number similarity is the principle that two flows at the same Re will behave identically regardless of scale — this is why wind tunnel models at different scales must match the Re of the full-size aircraft.

10.3 Why k-ω SST Matters

The choice of turbulence model significantly affects accuracy. The SST model captures adverse pressure gradient flows and boundary layer separation more accurately than simpler models like k-ε. This makes it the preferred model for aerofoil analysis at moderate Reynolds numbers where boundary layer behaviour critically affects Cl and Cd predictions.